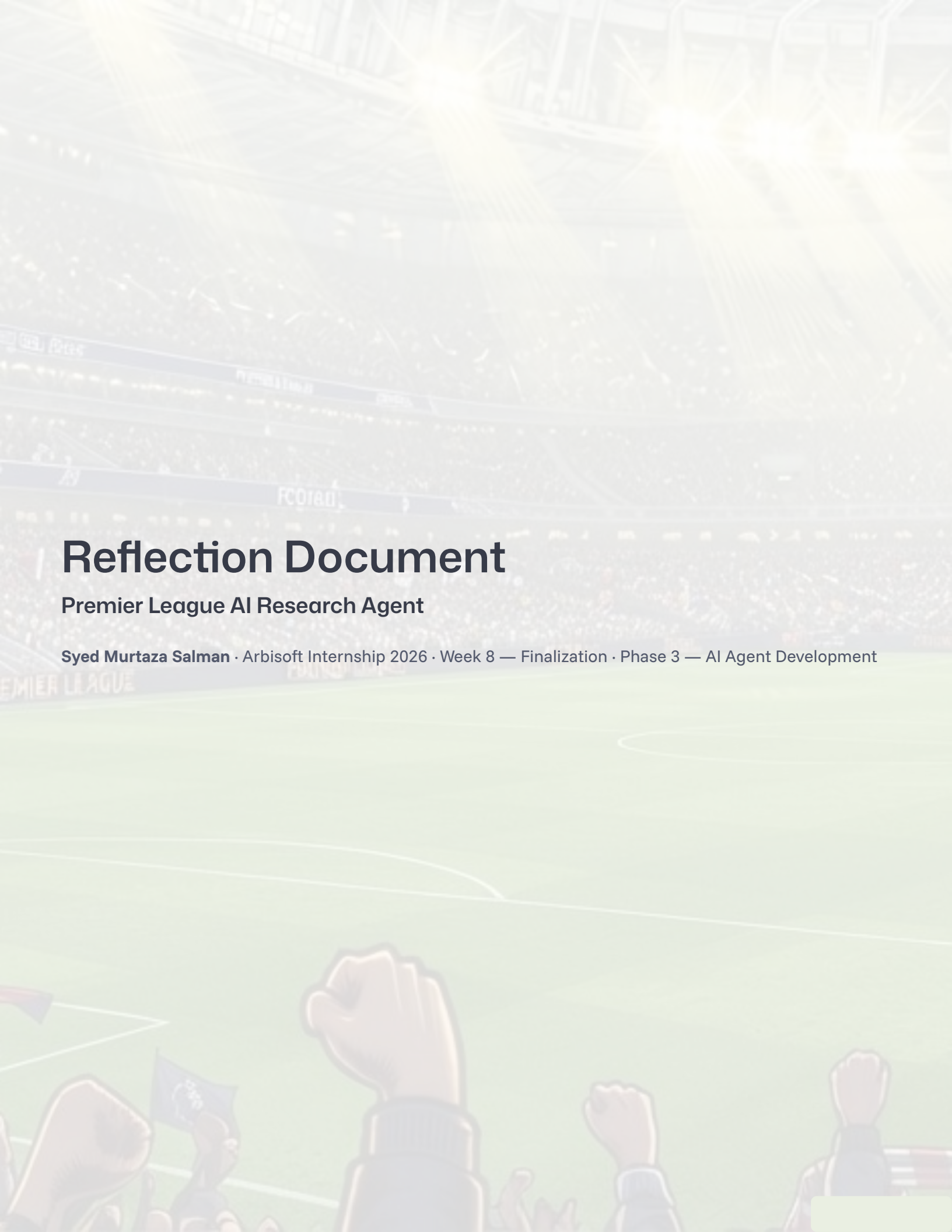




Reflection Document

Premier League AI Research Agent

Syed Murtaza Salman · Arbisoft Internship 2026 · Week 8 — Finalization · Phase 3 — AI Agent Development

Table of Contents

1. Introduction
2. What I Built
3. How AI Was Used During Development
4. What AI Did Well
5. Where AI Failed
6. What I Learned
7. Improvements Made During Week 8
8. What I Would Improve Further
9. Overall Reflection
10. Final Status
11. Evidence-Based Completion Status

1. Introduction

During Phase 3, I developed and refined a Premier League AI research assistant capable of answering football-related questions using live data, historical data, retrieval-augmented generation (RAG), external search, memory, multiple AI models, and specialized tools.

The objective was not simply to build a chatbot that could answer football questions. The goal was to build an AI agent that could identify the type of question being asked, select an appropriate tool, retrieve reliable information, and then generate an answer based on the retrieved data.

The project evolved considerably throughout the phase. Early work focused on establishing the core agent and tool infrastructure, while later weeks focused on RAG, historical statistics, multi-model routing, MCP integration, evaluation, error handling, and final user-facing polish.

2. What I Built

The final system combines several components, described below.

Live Premier League Tools

The agent can retrieve information such as:

- Current Premier League standings
- Current top scorers
- Recent results
- Upcoming fixtures
- Team-specific matches
- Transfer and football news

Historical RAG System

I implemented a historical data pipeline using ChromaDB. The historical match database was successfully populated with 4,157 Premier League matches. The system created a `pl_history` collection and stored the historical match documents with metadata such as:

- Season
- Competition
- Matchday
- Home team
- Away team
- Winner

I also implemented a separate player-statistics collection for historical player information. This allows the agent to answer questions about historical players and seasons rather than relying exclusively on the language model's internal knowledge.

Player Statistics

The player-statistics system loads player-season information and stores information such as:

- Games
- Minutes
- Goals
- Assists
- Yellow cards
- Red cards

The RAG system can then retrieve player statistics and generate career or season-level summaries.

Multi-Model Architecture

I implemented a primary model and a local fallback model. This means that if the primary model is unavailable or fails, the system can fall back to a locally running model rather than completely failing. The notebook includes a dedicated multi-model routing demonstration comparing the primary OpenRouter model with the local Ollama fallback.

MCP Server

I also implemented an MCP server demonstration exposing Premier League information through standardized resources and tools. The MCP implementation exposes resources including:

- `pl://standings`
- `pl://scorers`
- Tools for retrieving structured Premier League information

3. How AI Was Used During Development

AI pair programming was used throughout the development process. I used AI to help:

- Design the agent architecture
- Implement tool-calling logic
- Develop the RAG pipeline
- Debug Python errors
- Improve ChromaDB integration
- Develop player-statistics retrieval
- Improve query classification
- Develop fallback behavior
- Improve prompts
- Generate testing ideas
- Improve Markdown/table output
- Debug imports and environment issues
- Refine the interactive experience

However, the AI-generated code was not treated as automatically correct. I reviewed, executed, tested, and modified the generated code. Several issues discovered during development required manual debugging and redesign.

- ❏ This became one of the most important lessons of the project: AI pair programming significantly increases development speed, but the developer remains responsible for understanding and validating the resulting system.

4. What AI Did Well

AI was particularly effective in several areas.

4.1 Rapid Code Generation

AI significantly reduced the time required to create repetitive implementation code. For example, functions for:

- Data loading
- ChromaDB ingestion
- Metadata creation
- Tool routing
- Markdown formatting
- Evaluation loops

could be developed much faster with AI assistance.

4.2 Debugging Assistance

AI was useful when interpreting errors and identifying likely causes. During development I encountered issues including:

- `NameError`
- `ImportError`
- Missing ChromaDB collections
- Incorrect collection names
- Undefined OpenRouter configuration variables
- Mismatched imports
- Historical-query filtering problems

These failures became opportunities to understand how the project components were connected.

4.3 Prompt and Agent Design

AI was particularly useful for improving the agent's system prompt. The final prompt explicitly tells the agent which tools should be used for different categories of questions, such as:

Question Category	Tool Used
Historical statistics	RAG
Current standings	Standings tool
Current scorers	Scorer tool
Fixtures	Fixture tool
Recent results	Results tool
News / transfers	Web search

The evaluation framework also tests whether the agent selects the expected tool.

4.4 Generating Testing Ideas

AI helped identify different categories of questions that should be tested. The evaluation framework includes questions covering live information, historical statistics, news, team results, and other football queries.

5. Where AI Failed

AI assistance was useful, but it also introduced several problems.

5.1 Incorrect Assumptions About Existing Code

One major problem was that AI sometimes assumed that a variable, function, or object already existed. For example, I encountered:

```
NameError: name 'OPENROUTER_BASE_URL' is not defined
```

and:

```
ImportError: cannot import name 'chroma_client'
```

These errors showed that generated code can depend on assumptions that are not actually true in the current project.

5.2 ChromaDB Collection Problems

At one point the historical search returned:

```
Collection [pl_history] does not exist
```

The problem was not necessarily the search function itself; the underlying ChromaDB collection had not been created in the environment being queried. I had to inspect the ingestion pipeline, create the collection, and verify that the historical data was actually stored.

Eventually the ingestion process successfully stored:

```
4157/4157
```

```
Done.
```

```
Documents: 4157
```

and the ChromaDB database contained:

```
pl_history
```

```
pl_player_stats
```

This was an important debugging milestone.

5.3 Data Schema Mismatches

Another problem occurred when the search logic expected metadata fields that did not match the actual player-statistics data. This produced situations where a query such as a historical player search returned no results even though the underlying database contained player information.

This taught me that RAG quality depends heavily on consistent data schemas, not just on the quality of the embedding model or LLM.

5.4 AI Can Produce Technically Valid but Architecturally Wrong Code

Another lesson was that code can look correct in isolation while still being wrong for the overall project. For example, changing an import or function name can solve one immediate error while breaking another component that depends on the original interface.

This made me more careful about making changes at the architectural level rather than blindly accepting individual AI suggestions.

6. What I Learned

The biggest lessons from Phase 3 were not only about football data or LLMs, but about building reliable AI systems.

1 Tool Selection Is Critical

An AI agent is much more reliable when it retrieves information from the correct source rather than trying to answer everything from the language model. The evaluation framework explicitly tests whether questions are routed to the correct tools.

2 RAG Requires Good Data Engineering

Adding embeddings alone does not guarantee good retrieval. The data needs:

- Consistent metadata
- Useful document text
- Correct collection names
- Correct filtering
- Appropriate chunking
- Correct query interpretation

The historical RAG implementation taught me that retrieval quality depends on the entire pipeline.

3 Fallback Systems Improve Reliability

The multi-model architecture demonstrated that relying on one model can create a single point of failure. Having a local fallback model provides another path for producing an answer when the primary model fails.

4 Evaluation Should Be Built Into the Project

Instead of testing the agent only manually, I implemented evaluation questions that check:

- Whether the correct tool was selected
- Whether an answer was produced
- Whether important facts appeared in the answer

This makes the system easier to measure and improve.

5 AI Is a Development Partner, Not an Authority

The most important lesson was that AI-generated code must be reviewed. The AI helped me move faster, but it also created errors that I had to identify and correct. My role changed from simply writing every line of code to:

design → prompt AI → review → test → debug → validate → integrate

This is a much more realistic workflow for AI-assisted software development.

7. Improvements Made During Week 8

During the final stage of the project, I focused primarily on stability, documentation, usability, and final presentation rather than introducing unnecessary new features.

I also improved the agent's ability to handle comparison-style questions. Examples include comparisons such as:

- Mohamed Salah vs Erling Haaland
- Different seasons of the same player
- Leicester City's title-winning season vs Liverpool's 2019–20 campaign
- Historical team comparisons

The system can represent structured results using Markdown tables, making comparisons easier to understand. The interactive environment also supports Markdown rendering so that structured responses can be displayed as readable tables rather than raw Markdown text.

8. What I Would Improve Further

If I had additional development time, I would focus on the following areas.

Better Query Understanding

The current system uses a combination of LLM routing and query detection. A dedicated intent-classification layer could make routing even more reliable.

More Robust Historical Retrieval

Historical queries could be improved with stronger metadata filtering for:

- Player
- Team
- Season
- Date
- Competition

Better Comparison Generation

Comparison questions could automatically produce standardized tables with:

Category	Player / Team A	Player / Team B	Difference
—	—	—	—

This would make the system more consistent.

Better Deployment

A fully deployed web app would be a better option if I want to scale this idea to the bigger stage.

More Automated Testing

Future versions could include a larger automated regression suite so that changes to the RAG system or routing logic do not accidentally break previously working queries.

9. Overall Reflection

Overall, Phase 3 changed my understanding of AI development from simply using an LLM to building an actual AI-powered software system.

The most valuable part of the project was integrating multiple components into one system:

User → Agent → Tool Selection → Data Retrieval → AI Reasoning → Structured Answer

The project demonstrated that an effective AI application requires much more than a good language model. It requires reliable data sources, retrieval, tool integration, memory, fallback mechanisms, testing, validation, and a clear user interface.

AI pair programming made development substantially faster, especially for implementation and debugging. However, the errors introduced by AI also reinforced the importance of human review and testing.

- ❏ The biggest lesson I will take forward is that AI can accelerate software development, but reliability comes from engineering discipline, testing, and understanding the system — not from blindly trusting generated code.

10. Final Status

At the end of Phase 3, the core Premier League AI research agent is functional and demonstrates:

Live football data retrieval	Historical RAG
Player statistics retrieval	Tool-based agent routing
Multi-model fallback	MCP integration
Evaluation/testing	Interactive querying
Structured Markdown responses	Historical and comparative analysis

The remaining work is primarily focused on completing the final documentation, presentation, demo recording, and final project packaging.